



PROJETO NOVO

Princípio central

Existem três camadas:

1. GLOBAL
~/claude/CLAUDE.md
→ COMO todos os projetos devem ser desenvolvidos
→ Agent Teams
→ QA
→ Security
→ Review
→ regressão
→ critérios de conclusão
2. PROJETO
/projeto/CLAUDE.md
/docs/*
→ O QUE este sistema é
→ regras
→ arquitetura
→ dados
→ fluxos
→ comandos
3. TAREFA ATUAL
Sua mensagem para o Claude
→ O QUE queremos fazer agora

Seu papel passa a ser principalmente:

- VOCÊ
- └─ fornece contexto
 - └─ responde decisões de produto

- | aprova arquitetura
- | muda prioridades quando necessário
- | aprova publicação em produção

CLAUDE + AGENTES

- | analisa
- | documenta
- | propõe arquitetura
- | cria backlog
- | implementa
- | testa
- | audita
- | corrige
- | retesta
- | documenta

01 — INPUT

Responsável

Você.

Objetivo

Fornecer ao Claude tudo que representa a realidade do projeto.

Pode incluir:

- PDF
- MD
- DOC
- prints
- wireframes
- briefing
- documentos do cliente
- regras de negócio
- exemplos
- sistema antigo
- referências
- planilhas
- banco existente
- código existente

Idealmente coloque a matéria-prima em:

```
/projeto
├── docs/
│   └── fontes/
│       ├── briefing.pdf
│       ├── requisitos.md
│       ├── fluxo.pdf
│       ├── referencias/
│       └── ...
```

Esses arquivos são fontes. Não deixe o Claude modificá-los.

O que você faz

Só coloca os arquivos e informa onde estão.

Ainda **não peça para programar**.

02 — DISCOVERY

Responsável

Claude.

Objetivo

Entender profundamente o que precisa ser construído antes de escrever código.

Prompt padrão

Quero iniciar este projeto seguindo nosso workflow global de desenvolvimento.

FASE ATUAL: DISCOVERY.

Leia integralmente todos os materiais disponíveis em docs/ docs/archive docs/proposta docs/source e claude.md e também analise qualquer código existente neste repositório.

Não implemente nem altere código nesta etapa.

Quero que você identifique:

- objetivo do produto;
- usuários e perfis;
- módulos;

- funcionalidades;
- regras de negócio;
- fluxos;
- entidades e dados;
- permissões;
- integrações;
- requisitos funcionais;
- requisitos não funcionais;
- dependências externas;
- pontos contraditórios;
- informações ausentes;
- decisões que ainda precisam ser tomadas.

Não invente informações ausentes.

Quando algo não estiver definido nas fontes, marque explicitamente como PENDENTE DE DECISÃO.

Ao terminar a análise, avance para a geração da documentação inicial.

Você não precisa responder nada enquanto ele estiver apenas descobrindo.

03 — DOCUMENTAÇÃO INICIAL

Responsável

Claude.

Objetivo

Transformar PDFs, prints, MDs e outros materiais em documentação operacional.

Prompt padrão

Pode ser continuação automática da etapa anterior. Caso precise pedir:

Com base exclusivamente nas fontes analisadas e no que foi possível confirmar, gere a documentação inicial do projeto.

Crie:

```
docs/PRODUCT.md
docs/REQUIREMENTS.md
docs/FLOWS.md
docs/DATA-MODEL.md
```

```
docs/ARCHITECTURE.md
docs/ACCEPTANCE.md
docs/DECISIONS.md
```

Regras:

1. Não invente requisitos.
2. Diferencie claramente fatos confirmados, propostas técnicas e pendências.
3. ARCHITECTURE.md nesta etapa ainda é uma PROPOSTA, não uma arquitetura aprovada.
4. Em REQUIREMENTS.md, identifique os requisitos de forma rastreável.
5. Em ACCEPTANCE.md, crie critérios objetivos e testáveis.
6. Em DATA-MODEL.md, não invente campos sem necessidade identificada.
7. Registre todas as questões abertas para a próxima etapa.
8. Preserve os arquivos originais de docs/fontes/.

Resultado esperado

```
docs/
├── fontes/
├── PRODUCT.md
├── REQUIREMENTS.md
├── FLOWS.md
├── DATA-MODEL.md
├── ARCHITECTURE.md
├── ACCEPTANCE.md
└── DECISIONS.md
```

Aqui você **não precisa escrever esses documentos**.

O Claude faz.

04 — PENDÊNCIAS E DECISÕES

Responsável

Claude pergunta. Você decide.

Essa é uma das poucas etapas onde você precisa participar ativamente.

Prompt padrão

Agora faça a etapa de resolução de pendências.

Revise toda a documentação inicial e apresente SOMENTE decisões que realmente precisem da minha intervenção.

Não me pergunte decisões técnicas rotineiras que você consiga determinar com segurança pela arquitetura, boas práticas ou pelo próprio projeto.

Para cada decisão necessária, apresente:

1. contexto;
2. o que não está definido;
3. opções possíveis;
4. impactos de cada opção;
5. sua recomendação;
6. a pergunta objetiva que eu preciso responder.

Faça as perguntas em blocos organizados.

Depois das minhas respostas, registre automaticamente cada decisão em docs/DECISIONS.md e atualize os demais documentos afetados.

Exemplo

Claude:

DECISÃO 03 – Exclusão de clientes

- A) exclusão definitiva
- B) desativação
- C) arquivamento

Recomendação: B.

Motivo:
existem propostas e faturas relacionadas ao cliente.

Qual comportamento deseja?

Você:

B

Claude registra sozinho.

05 — ARQUITETURA

Responsável

Claude propõe. Você aprova.

Este é um **gate obrigatório**.

O Claude não deve começar o desenvolvimento antes da arquitetura estar aprovada.

Prompt padrão

Com os requisitos e decisões agora consolidados, proponha a arquitetura definitiva do sistema.

Ainda não implemente.

Defina conforme necessário:

- stack;
- organização do frontend;
- organização do backend;
- banco de dados;
- modelo de dados;
- autenticação;
- autorização;
- RBAC;
- APIs;
- integrações;
- armazenamento;
- filas/jobs;
- cache;
- segurança;
- observabilidade/logs;
- testes;
- infraestrutura;
- estratégia de deploy;
- estrutura de diretórios;
- fronteiras entre módulos;
- contratos importantes entre frontend e backend.

Não introduza tecnologias desnecessárias.

Priorize simplicidade, segurança, manutenção e velocidade de desenvolvimento.

Para decisões arquiteturais relevantes, explique brevemente o motivo.

Ao final, apresente:

ARQUITETURA RECOMENDADA
RISCOS
TRADE-OFFS
DECISÕES QUE AINDA PRECISAM DE MIM

Não implemente até eu aprovar.

Sua ação

Você lê e responde:

Arquitetura aprovada.

Ou:

Aprovado, mas quero PostgreSQL em vez de MySQL.

Isso é importante porque:

agentes paralelos aceleram uma arquitetura boa, mas também aceleram uma arquitetura ruim.

06 — DOCUMENTAÇÃO CONSOLIDADA

Responsável

Claude.

Depois da sua aprovação:

Arquitetura aprovada.

Consolide agora toda a documentação do projeto conforme a arquitetura aprovada e as decisões tomadas.

Atualize quando necessário:


```
PRODUCT.md
REQUIREMENTS.md
FLOWS.md
DATA-MODEL.md
ARCHITECTURE.md
ACCEPTANCE.md
DECISIONS.md
```

Remova das seções de pendência aquilo que já foi decidido.

Garanta consistência entre os documentos.

Ainda não comece a implementação.

Agora `ARCHITECTURE.md` passa a representar a arquitetura oficial.

07 — PROJECT CLAUDE.md

Responsável

Claude.

Você perguntou anteriormente se ele mesmo pode criar. **Sim, deve.**

Prompt padrão

Agora crie ou atualize o CLAUDE.md específico deste projeto.

O CLAUDE.md deve ser conciso e operacional.

Não copie toda a documentação para dentro dele.

Use os arquivos de docs/ como fonte detalhada e faça referências a eles.

Inclua somente informações persistentes que um agente precisa saber sempre que trabalhar neste sistema, incluindo quando aplicável:

- descrição resumida do projeto;
- documentação oficial;
- stack;
- estrutura principal;
- regras de negócio críticas;
- regras de segurança;
- RBAC;
- convenções importantes;

```
- comandos de desenvolvimento;  
- comandos de teste;  
- lint;  
- typecheck;  
- build;  
- migrations;  
- deploy;  
- coisas que nunca devem ser feitas.
```

O CLAUDE.md deste projeto deve complementar, e não substituir, minhas instruções globais.

Agora teremos:

```
GLOBAL  
/root/.claude/CLAUDE.md  
+  
PROJETO  
/projeto/CLAUDE.md
```

08 — BACKLOG

Responsável

Claude.

Você também **não precisa criar o backlog manualmente.**

Prompt padrão

```
Com documentação e arquitetura aprovadas, transforme o projeto em um backlog implementável.
```

```
Ainda não implemente.
```

```
Organize em:
```

```
EPICS
```

```
TASKS
```

```
SUBTASKS quando necessário
```

```
Para cada tarefa defina:
```

```
- ID;
```

- objetivo;
- responsável lógico;
- dependências;
- arquivos/módulos provavelmente envolvidos;
- critérios de aceite;
- testes necessários;
- riscos;
- possibilidade de execução em paralelo.

Monte também um grafo lógico de dependências.

Priorize uma ordem que permita colocar o máximo possível de trabalho independente em paralelo sem criar conflitos de edição ou dependências artificiais.

Identifique explicitamente:

PODE RODAR EM PARALELO
PRECISA AGUARDAR
BLOQUEIA OUTRA TAREFA

Salve o backlog em docs/BACKLOG.md.

Não implemente ainda.

Pode sair algo como:

```
EPIC 01 – FUNDAÇÃO
T001 Banco
T002 Autenticação
T003 RBAC

EPIC 02 – CLIENTES
T004 API
T005 Listagem
T006 Perfil

EPIC 03 – PROPOSTAS
...
```

09 — IMPLEMENTAÇÃO

Responsável

Claude Lead + Agent Team.

Aqui começa seu modelo multiagente.

Prompt padrão

Para executar uma Epic:

Na pasta ERROS coloquei um print d atela. nessa tela crie filtros para o admin, ondelke e le possa ver quem esta atrasado, em dia cancelados e etc..
Faça agora a implementação acima.

Instruções a seguir.

- CLAUDE.md;
- documentação oficial em /docs;
- arquitetura aprovada;
- BACKLOG.md;
- critérios de aceite;
- nosso workflow global.

Atue como Lead e use Agent Team quando houver paralelização útil.

Distribua o trabalho entre os especialistas globais adequados.

Priorize execução paralela de tarefas independentes e mantenha dependências explícitas.

Evite que dois agentes modifiquem simultaneamente os mesmos arquivos ou módulos sem coordenação.

Execute a implementação completa da Epic.

Para aplicações web, frontend e QA devem validar os fluxos relevantes também no produto renderizado usando Playwright, e não apenas pela leitura do código.

Não considere a Epic concluída quando a implementação terminar.

Após a implementação, execute automaticamente:

1. QA funcional;
2. Security Review;
3. Final Review;
4. consolidação dos findings;
5. correções pelos agentes responsáveis;
6. regressão;
7. novo QA/Security quando necessário;
8. testes automatizados;
9. lint/typecheck quando aplicável;
10. build de produção.

Continue o ciclo de correção e reteste até não existirem problemas Critical/High nem bugs bloqueadores.

Ao final, apresente:

- o que foi implementado;
- testes executados;
- findings encontrados e corrigidos;
- riscos ou pendências restantes;
- status final da Epic.

Não faça deploy em produção sem minha autorização.

Versão Codex:

Quero agora que você comece a implementação da Epic 1 inteira, task por task, de forma meticulosa e seguindo padrões focados em YAGNI e KISS.

Implemente isso usando como sabedoria base e prioridade informacional:

- AGENTS.md;
- documentação oficial em /docs;
- arquitetura aprovada;
- BACKLOG.md;
- critérios de aceite;
- nosso workflow global.

Atue como Lead e use Agent Team quando houver paralelização útil. Para os subagentes que te ajudaram a paralelizar tarefas, limite-os a apenas Luna low/medium/high/xhigh ou Terra low/medium. Valores maiores de esforço ou o modelo Sol é algo que apenas será usado em você que é Lead.

Distribua o trabalho entre os especialistas globais adequados.

Priorize execução paralela de tarefas independentes e mantenha dependências explícitas.

Evite que dois agentes modifiquem simultaneamente os mesmos arquivos ou módulos sem coordenação.

Execute a implementação completa da Epic.

Para aplicações web, frontend e QA devem validar os fluxos relevantes também no produto renderizado usando Playwright, e não apenas pela leitura do código.

Não considere a Epic concluída quando a implementação terminar.

Após a implementação, execute automaticamente:

1. QA funcional;
2. Security Review;
3. Final Review;
4. consolidação dos findings;
5. correções pelos agentes responsáveis;
6. regressão;
7. novo QA/Security quando necessário;
8. testes automatizados;
9. lint/typecheck quando aplicável;
10. build de produção.

Continue o ciclo de correção e reteste até não existirem problemas

Critical/High nem bugs bloqueadores.

Ao final, apresente:

- o que foi implementado;
- testes executados;
- findings encontrados e corrigidos;

- riscos ou pendências restantes;
- status final da Epic.

Não faça deploy em produção sem minha autorização.

A ideia é você **não microgerenciar**:

```
Lead
├─ frontend-engineer
├─ backend-engineer
└─ outros necessários
```

O próprio Lead distribui.

E para uma TASK específica?

Você não precisa escrever tudo novamente.

Pode usar uma versão menor:

Implemente a TASK [ID] do BACKLOG.md conforme a documentação, arquitetura, critérios de aceite e nosso workflow global.

Atue como Lead.

Use os agentes globais necessários e paralelize o que for independente.

Não pare na implementação: execute QA, Security/Review quando aplicável, correções e regressão até a tarefa estar realmente concluída.

Para funcionalidades web, valide o comportamento real com Playwright quando aplicável.

Não faça deploy em produção.

Por exemplo:

Implemente a TASK T042 – Cadastro de vagas do BACKLOG.md conforme a documentação, arquitetura, critérios de aceite e nosso workflow global.

Atue como Lead.

Use os agentes globais necessários e paralelize o que for independente.

Não pare na implementação: execute QA, Security/Review quando aplicável, correções e regressão até a tarefa estar realmente concluída.

Para funcionalidades web, valide o comportamento real com Playwright quando aplicável.

Não faça deploy em produção.

E pronto.

10 — QA + SECURITY + REVIEW

Responsável

Agentes.

Idealmente você não precisa enviar um prompt novo.

Seu global já deve provocar:

IMPLEMENTAÇÃO

↓

QA + SECURITY + REVIEW

Mas se precisar iniciar manualmente:

A implementação desta etapa terminou.

Inicie a fase de verificação independente conforme nosso workflow global.

Use em paralelo quando possível:

qa-engineer
security-reviewer
final-reviewer

QA deve validar o comportamento real e os critérios de aceite.

Security deve auditar os controles de segurança relevantes.

Final Reviewer deve comparar requisitos, arquitetura e implementação.

Nesta primeira passada, os revisores devem reportar findings ao Lead em vez de modificar código pertencente aos implementadores.

11 — CORREÇÕES

Responsável

Lead + agentes responsáveis.

Você também não deveria ficar copiando erros entre agentes.

Prompt, caso necessário

Consolide todos os findings de:

QA
Security
Final Review

Classifique por severidade e impacto.

Associe cada finding ao agente/módulo responsável.

Distribua as correções aos respectivos implementadores.

Prioridade:

1. Critical
2. High
3. bugs funcionais
4. regressões
5. Medium
6. Low

Não considere um finding resolvido apenas porque o código foi alterado.

Depois das correções, avance automaticamente para regressão.

12 — REGRESSÃO

Responsável

QA + Security quando aplicável.

Prompt padrão

Todas as correções desta rodada foram implementadas.

Execute agora regressão independente.

QA deve repetir:

- cenários que falharam;
- fluxos diretamente afetados;
- principais fluxos adjacentes;
- critérios de aceite da Epic.

Security deve verificar novamente cada finding de segurança corrigido.

Execute também os testes automatizados relevantes.

Se novos problemas forem encontrados, volte automaticamente para a etapa de correção.

Continue o ciclo:

finding
→ correção
→ regressão

até não restarem problemas Critical/High nem bugs bloqueadores.

13 — RELEASE CANDIDATE

Responsável

Claude.

Ainda **sem publicar produção.**

Prompt padrão

Prepare uma Release Candidate desta etapa/projeto.

Não faça deploy em produção ainda.

Execute tudo que for aplicável ao projeto:

- testes unitários;
- integração;
- E2E;
- lint;
- typecheck;
- build de produção;
- migration check;
- validação de configuração;
- verificação de variáveis obrigatórias;
- análise de logs;
- verificação de dependências;
- confirmação dos critérios de aceite.

Apresente um relatório final contendo:

IMPLEMENTADO
TESTADO
QA
SECURITY
BUILD
MIGRATIONS
RISCOS CONHECIDOS
PENDÊNCIAS
STATUS PARA RELEASE

Retorne:

READY FOR DEPLOY

somente se realmente estiver apto.

Não publique até minha autorização.

14 — APROVAÇÃO DE DEPLOY

Responsável

Você.

Esse é outro **gate humano obrigatório**.

Se estiver tudo certo:

Release aprovada.

Pode executar o deploy em produção conforme o procedimento documentado.

Após o deploy, execute smoke test e valide logs.

Eu manteria essa autorização sempre, principalmente porque você trabalha com servidores e bancos reais.

15 — DEPLOY + SMOKE TEST

Responsável

Claude.

Depois da autorização:

Execute o deploy aprovado.

Após a publicação:

1. confirme que os serviços subiram corretamente;
2. valide migrations aplicadas;
3. verifique logs;
4. execute smoke tests dos fluxos críticos;
5. valide autenticação;
6. valide banco/conectividade;
7. confirme APIs e integrações relevantes;
8. confirme que não existem erros críticos de runtime.

Se houver falha relevante, interrompa e reporte antes de realizar ações destrutivas ou rollback não autorizado.

16 — DOCUMENTAÇÃO FINAL

Responsável

Claude.

Muito importante para você conseguir administrar vários sistemas depois.

Prompt padrão

Com o deploy validado, finalize a documentação do projeto.

Atualize para refletir exatamente o estado atual da implementação:

PRODUCT.md
REQUIREMENTS.md
FLOWS.md
DATA-MODEL.md
ARCHITECTURE.md
ACCEPTANCE.md
DECISIONS.md
BACKLOG.md
CLAUDE.md

Crie ou atualize também:

docs/RUNBOOK.md
docs/CHANGELOG.md

RUNBOOK.md deve explicar quando aplicável:

- como iniciar;
- como parar;
- como reiniciar;
- como fazer build;
- como fazer deploy;
- migrations;
- backups;
- restauração;
- logs;
- serviços;
- variáveis necessárias sem expor secrets;
- troubleshooting;
- health checks.

CHANGELOG.md deve registrar o que foi entregue nesta versão.

Marque no backlog as tarefas realmente concluídas.

Registre pendências reais que ficaram para versões futuras.

Não documente como existente algo que ainda não foi implementado.

O fluxo completo que você pode colocar na documentação

```
01 INPUT
  ↓
  VOCÊ fornece fontes

02 DISCOVERY
  ↓
  CLAUDE entende sem programar

03 DOCUMENTAÇÃO INICIAL
  ↓
  CLAUDE cria docs

04 PENDÊNCIAS
  ↓
  CLAUDE pergunta
  VOCÊ decide

05 ARQUITETURA
  ↓
  CLAUDE propõe
  VOCÊ aprova

06 CONSOLIDAÇÃO
  ↓
  CLAUDE atualiza documentação

07 PROJECT CLAUDE.md
  ↓
  CLAUDE cria

08 BACKLOG
  ↓
  CLAUDE cria tasks + dependências

09 IMPLEMENTAÇÃO
  ↓
  AGENT TEAM

10 VERIFICAÇÃO
  ↓
  QA + Security + Review

11 CORREÇÕES
  ↓
  Agents responsáveis
```

```
12 REGRESSÃO
  ↓
QA + Security novamente

13 RELEASE CANDIDATE
  ↓
Test + Lint + Typecheck + Build + Migration Check

14 APROVAÇÃO
  ↓
VOCÊ

15 DEPLOY
  ↓
CLAUDE
Smoke Test + Logs

16 DOCUMENTAÇÃO FINAL
  ↓
CLAUDE
```

O mais importante: você não precisa mandar 16 prompts em toda situação

Esse é o **procedimento completo**.

Para um projeto novo grande, vale seguir todas as etapas.

Para uma nova Epic em um sistema já documentado, você normalmente começa em:

```
08 Backlog
ou
09 Implementação
```

Para um bug:

```
Análise
→ implementação
→ QA
→ regressão
```

Para uma alteração visual pequena:

```
implementação  
→ verificação
```

Ou seja, **o método é fixo, mas o Lead deve aplicar somente as etapas necessárias para o tamanho da mudança.**

E eu acrescentaria uma regra ao seu documento

| Nunca acelerar pulando entendimento; acelerar paralelizando execução.

É isso que diferencia seu novo fluxo de simplesmente colocar cinco IAs escrevendo código ao mesmo tempo. O ganho vem de você ter **uma arquitetura aprovada + backlog com dependências + especialistas paralelos + validação independente.**